

LIVA: A Multi-Agent LLM-Assisted System for IoT Vulnerability Analysis

Zhe Yang , Hao Peng , Senior Member, IEEE, Yanling Jiang, Jianwei Liu , Senior Member, IEEE, Hongbin Luo , Mingsheng Tang, Member, IEEE, Jiahe Li , and Kun Zhang 

Abstract—IoT devices have become deeply integrated into our daily lives, making comprehensive security research on critical infrastructure devices increasingly important. Static analysis techniques, particularly those leveraging taint propagation, have demonstrated promise in identifying security vulnerabilities within these devices, effectively detecting critical vulnerabilities. However, current solutions often struggle with limitations in both detection efficiency and accuracy. To address these challenges, this paper introduces Liva, a novel static taint analysis tool designed for detecting web vulnerabilities in IoT devices. Liva employs a large language model (LLM) multi-agent approach for static binary taint analysis, primarily leveraging fine-tuned open-source models and commercial LLMs to improve source/sink identification and taint data analysis—areas where traditional methods often fall short—thereby enhancing overall analysis efficiency. LIVA’s core analysis engine leverages a Qwen3-32B open-source model that has been fine-tuned using a dataset of 3,000 real-world device samples. This fine-tuned model achieves a 3 percentage point improvement in accuracy for identifying taint data propagation relationships compared to commercial LLMs, while also increasing average analysis efficiency by 5.5%. A comprehensive evaluation of Liva, conducted on a dataset of 64 devices from 11 vendors, revealed that it detected 309 and 349 more known vulnerabilities than the state-of-the-art solutions SaTC and Karonte, respectively, while simultaneously reducing false positive rates by 59.4% and 67.6%. Liva achieves a recall of 98.1% and a precision of 74.6%, with a $6.7\times$ reduction in analysis time compared to the best-performing baseline. Furthermore, in the realm of zero-day vulnerability detection, Liva discovered 64 previously unknown vulnerabilities, 39 of which have since been assigned official CVE/CNVD identifiers.

Index Terms—Internet of Things, taint analysis, large language models, vulnerability, fine-tuning.

I. INTRODUCTION

THE rapid development of Internet of Things (IoT) technologies is changing human life. The number of globally connected devices has surpassed 10 billion and is projected to

reach 27 billion by 2025 [1]. This expansion has led to a critical network infrastructure comprising routers, Wi-Fi access points, VPN gateways, and firewalls. These edge devices, responsible for data routing, encryption, and security, are directly exposed to public networks, rendering them prime targets for cyberattacks. The increasing prevalence of these devices presents significant security challenges for embedded systems. Check Point’s “2025 Cybersecurity Report” indicates a 44% increase in global cyberattacks since 2023, with 37% specifically targeting edge devices and network infrastructure [2]. Edge devices, particularly routers, VPN gateways, and IoT devices, have become crucial entry points for attackers [3], underscoring the essential nature of security research focused on IoT critical devices.

Within the domain of IoT device security analysis, taint analysis is particularly critical due to three fundamental characteristics of the firmware ecosystem: its closed-source nature, inherent hardware dependency, and the prevalence of specific vulnerability patterns. Most commercial IoT devices rely on closed-source firmware, which hinders effective dynamic simulation [4], [5]. For instance, FirmAE’s [6] network service simulation for Linux firmware achieves only a 79% success rate [6], thereby limiting the applicability of dynamic fuzzing techniques. Moreover, IoT firmware vulnerabilities often exhibit taint-like behavior [7], [8], where unverified external inputs (e.g., network packets) propagate along data-dependent paths to sensitive operation points such as memory manipulation functions or command execution interfaces [9]. In this context, static binary taint analysis offers notable advantages: by directly parsing the firmware’s instruction stream, it establishes cross-function data-flow relationships without requiring source code or hardware emulation, enabling the tracing of complete paths from taint sources (e.g., `websGetVar`) to sensitive sinks (e.g., `strcpy`) [10], [11]. Several advanced tools, including KARONTE [12], SaTC [13], and Emtaint [14], have demonstrated greater practicality than dynamic solutions in analyzing IoT firmware security. Nevertheless, existing static taint analysis approaches still face several limitations that hinder their effectiveness in real-world IoT scenarios.

Identifying source and sink points is challenging, often causing false negatives. The diversity of input sources in IoT devices and the unpredictable nature of their routing mechanisms complicate the precise identification of source points [15]. Furthermore, the encapsulation of potentially dangerous sink points within shared libraries hinders their recognition and subsequent handling [16], [17].

Received 9 September 2025; revised 16 December 2025; accepted 12 February 2026. Date of publication 16 February 2026; date of current version 12 May 2026. This work was supported by the Beijing Natural Science Foundation under Grant L253021, in part by NSFC under Grant 62322202 and Grant U25B2029, in part by the Local Science and Technology Development Fund of Hebei Province Guided by the Central Government of China under Grant 254Z9902G, and in part by Shijiazhuang Science and Technology Plan Project under Grant 2511301807A. (Corresponding author: Hao Peng.)

The authors are with the School of Cyber Science and Technology, Beihang University, Beijing 100191, China (e-mail: zheyang.yz@foxmail.com; penghao@buaa.edu.cn; yanlingj@buaa.edu.cn; liujianwei@buaa.edu.cn; luohb@buaa.edu.cn; tangmingsheng@buaa.edu.cn; hi_jacklee@buaa.edu.cn; zhangkun9605@foxmail.com).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TDSC.2026.3665343>, provided by the authors.

Digital Object Identifier 10.1109/TDSC.2026.3665343

Security analysis of IoT devices faces low efficiency in tainted data propagation analysis. The complex function call relationships often lead to significant redundant computations when independently analyzing each source function and its corresponding sink function [18]. Additionally, individual functions may contain multiple sensitive invocation points, and traditional methods that analyze each invocation point separately not only waste computational resources but also significantly reduce the overall analysis efficiency [19].

Another significant limitation of traditional taint analysis stems from its reliance on static rules. Existing approaches, such as SaTC, Karonte, and EmTaint, primarily depend on predefined taint propagation rules and pattern-matching techniques. However, for IoT devices with diverse architectures and complex component interactions, static rules often fail to encompass all possible taint propagation paths, leading to both high false-positive and false-negative rates [20].

To address these limitations, we propose Liva, an LLM-driven taint analysis system specifically designed for IoT firmware vulnerability detection, and we have implemented a prototype consisting of over 9,000 lines of code, integrating IDA Pro [21], Ghidra [22], and specialized LLM agents. Liva overcomes the limitations of traditional taint analysis methodologies by leveraging fine-tuned large language models for nuanced semantic code comprehension and employing graph databases to enable efficient storage and querying of taint flows. Liva introduces three key innovations to address the aforementioned challenges:

Enhanced Source and Sink Identification: we incorporate an enhanced identification algorithm that analyzes firmware strings to extract function names and decompiled code that invokes them. Then Liva uses an LLM to determine which functions possess user input capabilities, classifying them as source points. For dangerous sink points within shared libraries, Liva performs more in-depth analysis of their encapsulation methods, enabling a comprehensive identification strategy applicable across diverse scenarios.

Clustering-based Efficiency Optimization: we propose a novel method for analyzing dangerous function flows based on source and sensitive function clustering. This approach groups source functions that share the same processing function into clusters. For each source function cluster, Liva employs a unified exploration strategy to traverse the function call graph using search algorithms, starting from all source functions and generating a set of call paths leading to potential sinks. During taint analysis, functions within the same chain of function calls that share common invocations are grouped together for unified taint data propagation analysis.

Multi-agent LLM System: we introduce a novel multi-agent taint analysis System based on fine-tuned large language models. By implementing a taint propagation agent and a vulnerability inference agent, Liva overcomes the constraints of traditional static rules. The taint propagation agent's LLM, fine-tuned for domain-specific knowledge, not only has a deep understanding of code semantics but also dynamically adapts to diverse taint propagation patterns and parameter-passing relationships. The vulnerability inference agent then leverages these analysis results to determine the presence of actual security risks, thereby improving accuracy while simultaneously reducing false positives.

To evaluate Liva's effectiveness, we conducted comprehensive experiments using a dataset comprising 64 firmware samples collected from 11 prominent IoT vendors. This dataset was constructed by combining the established SaTC and Karonte datasets with 15 additional new firmware samples. Extensive benchmarking was then conducted against state-of-the-art firmware analysis tools, specifically Karonte and SaTC, using this comprehensive 64-device dataset. The evaluation results underscore Liva's superior vulnerability detection capabilities, demonstrating the identification of 309 additional known vulnerabilities compared to SaTC and 349 more than Karonte. Furthermore, Liva exhibits enhanced reliability, achieving false positive rate reductions of 59.4% and 67.6% relative to these baselines, respectively. The system maintains robust performance metrics, achieving a recall of 98.1% and a precision of 74.6%, while simultaneously delivering exceptional efficiency, completing analysis 6.7 times faster than the best-performing baseline. Beyond the detection of known vulnerabilities, real-world validation of Liva revealed 64 previously undisclosed security flaws, with 39 successfully obtaining official CVE/CNVD designations.

The key contributions of Liva are summarized as follows:

- We propose an LLM-assisted methodology for precisely identifying taint sources and sinks.
- We use Clustering-based optimization, which significantly enhances efficiency in taint propagation analysis.
- We are the first to leverage both fine-tuned and commercial LLMs to build a multi-agent system dedicated to taint propagation analysis and subsequent vulnerability detection.
- We present a comprehensive evaluation of Liva using 64 firmware datasets, discovering 370 vulnerabilities including 64 previously unknown, with 39 officially reported and certified.

We make the firmware dataset, detailed documentation, source code, and a collection of fine-tuning examples publicly available at <https://github.com/RingBDStack/LIVA>.

II. BACKGROUND

A. Threat Model

This study investigates taint-related vulnerabilities within web services of IoT network node devices, including routers, VPN gateways, wireless access points, network repeaters, and network storage devices. The firmware on these devices typically operates on Linux-based hardware, and their backend web services are often supported by binary programs [23].

Our threat model assumes that an attacker can acquire the target device's firmware and perform reverse engineering to understand the implementation details of the backend services. The attacker can access the device's web service via LAN or the Internet and send malicious HTTP requests [24]. IoT web services generally provide numerous input entry points, such as login forms, configuration pages, and file upload functionalities. We treat these entry points as potential taint sources that may introduce untrusted data, which can propagate to unsafe backend functions and cause vulnerabilities.

B. Taint Analysis

Taint analysis is a critical program security analysis technique used to track the propagation paths of potentially hazardous data, enabling the identification and prevention of security vulnerabilities. The taint analysis process typically comprises three key stages: taint source identification, sensitive sink location, and taint propagation analysis. However, existing solutions still encounter several challenges.

Source Identification: This process aims to locate functions or operational points within the system that receive external input, which could potentially be manipulated by attackers and lead to security vulnerabilities. In IoT device web services, such input is typically obtained from users via the HTTP protocol. SATC [10] uses common input keywords between front-end and back-end to identify taint sources, but cannot detect sources lacking frontend code. LARA [25] accurately describes taint sources using URIs and keys, but fails to identify sources in IoT devices' special routing methods.

Sink Identification: This refers to identifying dangerous function call points that perform sensitive operations, such as string manipulation (e.g., `strcpy`, `strcat`), command execution (e.g., `system`, `execl`), and formatted output (e.g., `printf`, `fprintf`). SATC and Karonte use predefined sinks but cannot analyze sinks in shared libraries (.so) or support incremental sink analysis.

Tainted Data Propagation Analysis: Taint analysis traces the propagation of potentially untrusted data from source points through the program to determine whether it eventually reaches sensitive sink points. LATTE [26] is the first study to apply LLM to taint analysis. It leverages prompt-based interpretation of binary code snippets to assess the connectivity of tainted data. However, LATTE does not reuse intermediate analysis results, leading to increased computational overhead.

C. Large Language Model

Large Language Models (LLMs) are advanced neural network architectures trained on large collections of text and code. They can capture complex syntax, semantics, and contextual patterns [27]. By using pre-training on domain-specific datasets followed by fine-tuning, LLMs achieve strong performance in natural language understanding, logical inference, and code semantic analysis [28]. In software security analysis, LLMs can explain low-level program semantics, infer data flow relationships, and identify potential vulnerability patterns [29]. Their ability to generalize across different instruction set architectures and diverse coding styles makes them particularly well-suited for binary taint analysis [30].

D. Motivation

Fig. 1 shows a function code segment in the binary firmware of a D-Link router that contains a command injection vulnerability. The trigger process is as follows: after receiving an HTTP request, the backend binary file `jhttpd` uses route matching to direct the program flow to the `usb_passwd_asp` function. This function first reads the `name` value from the HTTP request

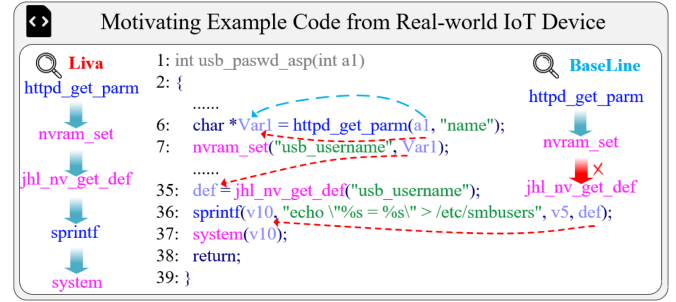


Fig. 1. Motivating example.

and calls `nvram_set` to store it in the configuration. Then, at line 35, it calls `jhl_nv_get_def` to read the `name` value from the configuration again. At line 36, it assigns the value to variable `v10` using `sprintf`, and finally, at line 37, it executes `system`, which triggers the command injection.

In our vulnerability detection experiment, we used SaTC, KARONTE, and EmTaint for taint analysis. None of them could detect the complete parameter transfer chain. The reason is that these tools rely on fixed variable propagation rules, which cannot fully parse how variables are processed, where they are stored, and how they are later retrieved in the code. Such traditional static-rule-based analysis methods perform poorly when the taint propagation path is complex.

As shown in Fig. 1, the right side presents the results from traditional tools, where the analysis stops once the variable is passed to `nvram_set`, because these tools cannot interpret its actual function and thus fail to continue tracing.

In contrast, the left side shows analysis by Liva, which, by recognizing the “store–retrieve” semantic relationship between `nvram_set` and `jhl_nv_get_def`, can associate the `def` parameter obtained by `jhl_nv_get_def` with the earlier `var1`, thereby discovering the vulnerability.

III. CHALLENGE

A. Accurate Source and Sink Identification

IoT firmware presents challenges due to diverse hardware architectures (ARM, MIPS, x86) and heterogeneous input handling mechanisms [31]. Traditional expert-defined source/sink identification lacks scalability across firmware variants, while frontend-backend string matching approaches suffer from high false positive rates due to parsing ambiguities.

B. Efficient Taint Analysis

IoT firmware binaries contain extensive function call graphs with significant path overlap, where multiple vulnerability sinks share common intermediate processing functions [32]. Naive taint analysis introduces computational redundancy through repeatedly traversing identical code paths. The key challenge is to develop clustering strategies that merge related sources, sinks, and intermediate functions, thereby minimizing redundant analysis while preserving detection accuracy.

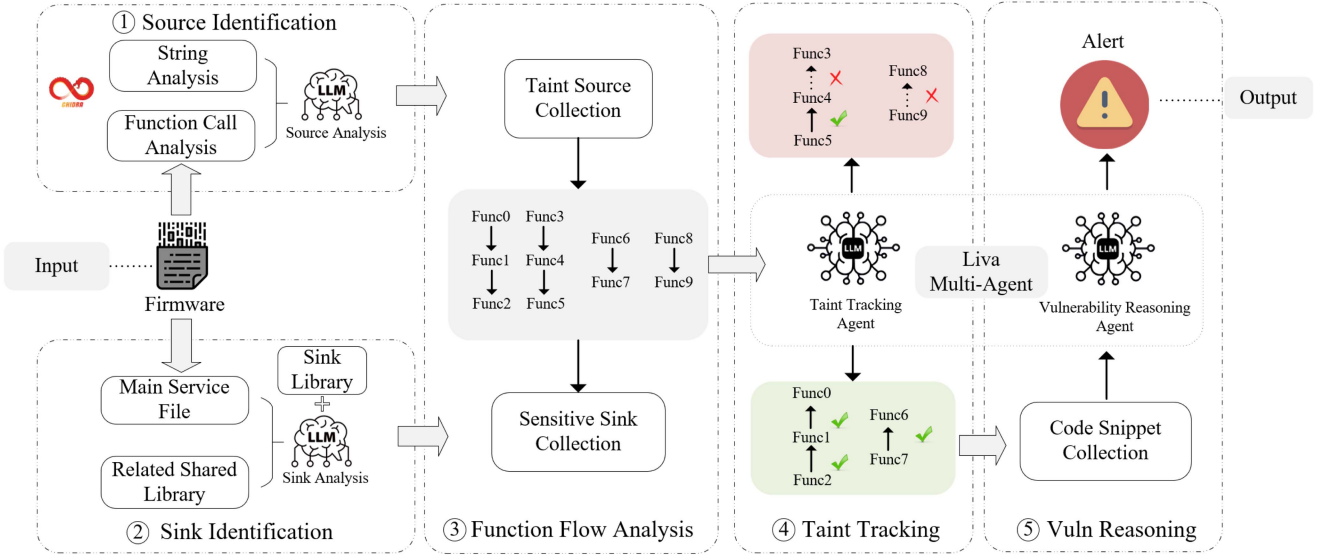


Fig. 2. The overview of Liva.

C. LLM Application Challenges

Applying large language models to taint analysis in IoT firmware introduces two critical challenges. First, token limitations constrain the analysis of large-scale decompiled code, often requiring truncation that compromises analysis completeness [33], [34]. This issue is particularly pronounced when analyzing complex firmware characterized by extensive function call graphs and deep nesting structures. Second, achieving standardized output formats suitable for automated processing requires domain-specific fine-tuning rather than prompt engineering alone [35]. The variability in LLM responses introduces parsing inconsistencies that undermine the reliability of downstream analysis. Constructing high-quality training datasets that accurately capture taint propagation semantics across diverse firmware implementations remains a fundamental challenge for the effective application of LLMs in taint analysis. Such datasets must encompass the full spectrum of parameter passing patterns and vulnerability manifestations encountered in real-world IoT systems.

IV. DESIGN OF LIVA

To address these challenges, we propose Liva, an LLM-based system for IoT vulnerability analysis. As shown in Fig. 2, Liva adopts a modular architecture that transforms firmware binaries into vulnerability reports through five stages: 1) Source Identification: This stage combines string-function recognition with LLM-powered semantic analysis to accurately locate external input entry points. 2) Sink Identification: Here, predefined vulnerability libraries are integrated with LLM-driven analysis to identify security-critical wrapper functions that represent potential vulnerability sinks. 3) Dangerous Function Flow Analysis: Code slicing techniques are applied to establish function-level connections between identified candidate sources and sinks. 4) Taint Propagation Tracking: A fine-tuned Liva agent is employed to trace data dependencies and determine the propagation paths

from source to sink. 5) Vulnerability Inference: Finally, the connected code segments and metadata are analyzed by an inference agent to classify vulnerabilities and assess their severity.

A. Source Identification

To automatically identify input sources in IoT firmware, we propose a backend-driven method that leverages constant strings in binaries in conjunction with the semantic analysis capability of LLMs. Unlike traditional approaches that rely on front-end code, our method directly analyzes the back-end binaries, which proves particularly effective in scenarios where the coupling between front-end and back-end is weak or where the front-end files are absent.

The method operates in three stages. 1) *String extraction*: constant strings $S = \{s_1, s_2, \dots, s_n\}$ are extracted from binary sections (e.g., `.rodata`). 2) *Reference localization*: for each string $s_i \in S$, we locate its references in the binary and identify the corresponding function calls in which s_i appears as an argument. Let the resulting candidate set be F . 3) *LLM-based classification*: each $f \in F$ is analyzed by the LLM to determine whether the call semantically represents an external input operation (e.g., HTTP parameter retrieval).

The final set of source functions is thus:

$$\mathcal{F}_{\text{source}} = \{f \in F \mid \text{LLM_classify}(f) = 1\} \quad (1)$$

B. Sink Identification

To achieve comprehensive detection of potentially dangerous function calls (sink points), we design a dual-level framework that analyzes both the *main program* and the *shared libraries*. This strategy enables us to capture both explicit risks and semantically wrapped risks that are indirectly introduced by library functions.

1) *Main Program Level (Direct Function Matching)*: In the main program, potentially dangerous function invocations are

identified through direct comparison with a predefined set of hazardous sinks, denoted as $\mathcal{D}_{\text{sink}}$ (e.g., `system`, `popen`, `strcpy`). Any function call that exactly matches an entry in $\mathcal{D}_{\text{sink}}$ is immediately classified as a sink, thereby enabling rapid and precise detection of explicit vulnerable call sites.

2) *Shared Library Level (Wrapper Function Identification)*: IoT firmware extensively uses shared libraries whose exported functions often wrap dangerous operations. We employ a two-stage approach to detect these indirect risks.

First, we collect the set of exported functions from all shared libraries referenced by the main program, and apply lightweight heuristics—such as functions involving pointer or buffer arguments, containing nested calls, or including suspicious keywords like `exec`, `auth`, or `parse`—to filter a candidate subset. Second, for each candidate function l , we construct its control flow graph and perform taint propagation with LLM. If the taint analysis reveals a path from l to any known dangerous sink $d \in \mathcal{D}_{\text{sink}}$, then l is classified as a wrapper sink function. This mechanism allows us to identify indirect sink points that are not visible at the main program level but are exposed through shared library abstractions.

3) *Combined Sink Identification*: The final sink set $\mathcal{F}_{\text{sink}}$ combines direct sinks from the main program and wrapper sinks from shared libraries, serving as the foundation for taint propagation and vulnerability discovery.

C. Taint Analysis Optimizations

1) *Source-Clustering-Based Function Flow Analysis*: To improve the efficiency and coverage of inter-procedural taint analysis, we propose a top-down function flow analysis method based on *source clustering*. By grouping related source functions into clusters and jointly exploring their call paths, redundant traversals are reduced while ensuring comprehensive coverage of potential sinks.

Let $\mathcal{F}_{\text{source}}$ denote the complete set of identified source functions. In our framework, these sources are clustered through the procedure `CLUSTER`($\mathcal{F}_{\text{source}}$), which groups source functions according to their parent functions in the call graph. For each source function f_s , we obtain its parent function $\text{parent}(f_s) = g$ where $(g \rightarrow f_s) \in E$, meaning that g is the direct caller of f_s . The `CLUSTER` procedure then aggregates all source functions sharing the same parent function into a single cluster, expressed as $C(g) = \{f_s \mid \text{parent}(f_s) = g\}$. The complete cluster set returned by `CLUSTER` is simply the collection of all non-empty $C(g)$. This parent-based grouping ensures that sources originating from the same enclosing function are analyzed jointly, thereby reducing redundant traversal across similar call paths.

We model the program as a call graph $G = (\mathcal{F}, E)$, where $\mathcal{F}$ is the set of functions and E the set of call edges. For each cluster $C_i \in \mathcal{C}_{\text{source}}$, we perform a joint top-down exploration from all sources in C_i . If a path from C_i reaches a sink function, the sink is recorded as reachable. Formally, the set of sinks discovered from cluster C_i is

$$\mathcal{F}_{\text{sink}}^{(i)} = \left\{ f^{(t)} \in \mathcal{F}_{\text{sink}} \mid \exists \text{ path from } C_i \text{ to } f^{(t)} \right\} \quad (2)$$

Algorithm 1: Cluster-Based Taint Flow Discovery.

Require: Call graph $G = (\mathcal{F}, E)$; source set $\mathcal{F}_{\text{source}}$; sink set $\mathcal{F}_{\text{sink}}$

Ensure: Dangerous flow set $\mathcal{P}_{\text{danger}}$

```

1: function DISCOVERFLOWS( $G, \mathcal{F}_{\text{source}}, \mathcal{F}_{\text{sink}}$ )
2:    $\mathcal{C} \leftarrow \text{CLUSTER}(\mathcal{F}_{\text{source}})$ 
3:    $\mathcal{P}_{\text{danger}} \leftarrow \emptyset$ 
4:   for all  $C \in \mathcal{C}$  do
5:     for all  $f \in C$  do
6:        $\mathcal{P}_f \leftarrow \text{FINDPATHS}(f, \mathcal{F}_{\text{sink}}, G)$ 
7:        $\mathcal{P}_{\text{danger}} \leftarrow \mathcal{P}_{\text{danger}} \cup \mathcal{P}_f$ 
8:     end for
9:   end for
10:  return  $\mathcal{P}_{\text{danger}}$ 
11: end Function

```

In other words, a single traversal starting from a source cluster can cover multiple call paths and sinks, thereby avoiding redundant per-source exploration. Algorithm 1 summarizes the cluster-based strategy. By sharing traversals across semantically related sources, the method improves the scalability of inter-procedural taint analysis in large-scale IoT firmware.

2) *Parameter Mapping Structure*: To address the fundamental challenge of accurately modeling inter-procedural data dependencies in static program analysis, we introduce a novel parameter mapping structure that formally captures the semantics of function call relationships. Unlike traditional approaches that rely on complex control flow graphs, our structure provides a focused abstraction that models parameter-passing semantics while maintaining efficiency.

Our approach represents each function call relationship as a structured tuple $\langle f_c, \mathcal{M}, a, c \rangle$ associated with parent function f_p , where $f_c \in \mathcal{F}$ denotes the called function, $\mathcal{M} : \mathcal{V}_{f_p} \rightarrow \mathcal{P}_{f_c}$ represents the parameter mapping from parent variables to child parameters, a specifies the call site address, and c contains the corresponding decompiled code fragment. Formally, our complete data structure can be expressed as:

$$\mathcal{D} = \{f_p \mapsto \langle f_c, \mathcal{M}, a, c \rangle \mid f_p \in \mathcal{F}_{\text{parent}}, f_c \in \mathcal{F}_{\text{child}}\} \quad (3)$$

where the mapping function $\mathcal{M} = \{v_p \mapsto p_c \mid v_p \in \mathcal{V}_{f_p}, p_c \in \mathcal{P}_{f_c}\}$ establishes precise correspondences between caller variables $\mathcal{V}_{f_p}$ and callee parameters $\mathcal{P}_{f_c}$. To illustrate this structure concretely, the parameter mapping can be represented in the following simplified notation: { `ParentFunction`: (`CalledFunction`, `ParamMapping`, `CallAddress`, `CallCode`) }

This parameter-passing relationship structure $\mathcal{M}$ completely preserves data flow path information, thereby enabling comprehensive taint propagation analysis. Most importantly, the *hierarchical composition* property allows our structure to naturally handle complex call chains through recursive application: for a call chain $f_1 \rightarrow f_2 \rightarrow \dots \rightarrow f_n$, the complete parameter flow can be computed as:

$$\mathcal{M}_{1,n} = \mathcal{M}_{n-1,n} \circ \mathcal{M}_{n-2,n-1} \circ \dots \circ \mathcal{M}_{1,2} \quad (4)$$

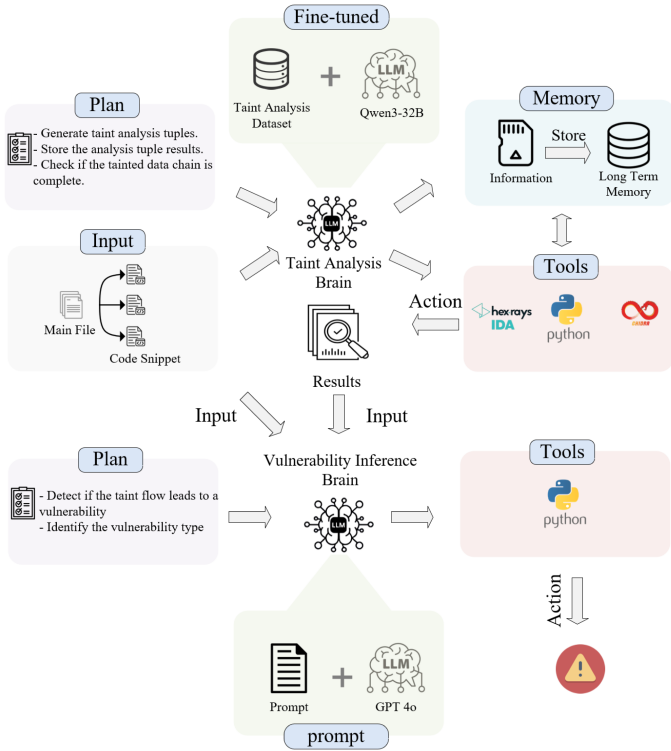


Fig. 3. Liva multi-agent framework. The framework consists of two main components: the taint analysis agent and the vulnerability inference agent.

This compositional property provides significant advantages for large-scale program analysis. The reusability of intermediate nodes allows frequent access to function parameter mapping $\mathcal{M}_{i,j}$, thus avoiding redundant subpath analysis and enabling concurrent analysis.

3) *Call Chains Merging*: A critical efficiency bottleneck in traditional inter-procedural analysis arises from the redundant processing of parent functions that invoke multiple child functions. To address this computational inefficiency, we introduce a *call site merging strategy* that consolidates analysis contexts and significantly reduces analytical overhead.

Consider a parent function $f_p \in \mathcal{F}_{\text{parent}}$ that invokes multiple child functions across different call sites. We formally define its invocation context as $\mathcal{C}(f_p) = \{(f_c, l) \mid f_c \in \mathcal{F}_{\text{child}}, l \in \mathcal{L}_{f_p}\}$, where each tuple (f_c, l) represents a specific function call to f_c at location l within the parent function's address space $\mathcal{L}_{f_p}$. Traditional approaches analyze each parent-child relationship (f_p, f_c) independently, resulting in $|\mathcal{C}(f_p)|$ separate traversals of the same parent function code, leading to significant computational redundancy.

Our optimization strategy eliminates this redundancy through a unified analysis framework. Rather than performing isolated analyses, we construct a merged analysis context:

$$\mathcal{M}(f_p) = \bigcup_{(f_c, l) \in \mathcal{C}(f_p)} \text{AnalyzeCall}(f_p, f_c, l) \quad (5)$$

where the function $\text{AnalyzeCall}(f_p, f_c, l)$ captures the parameter mapping and taint flow relationships for the specific

call instance from f_p to f_c at location l . This consolidation enables a single comprehensive traversal of f_p that simultaneously extracts all relevant call relationships, transforming multiple independent analyses into a unified processing workflow.

The practical impact of this optimization is particularly pronounced in firmware and embedded system codebases, where dense call structures and repetitive invocation patterns are prevalent. In our implementation, this merged analysis is further enabled through a fine-tuned large language model, which is capable of simultaneously interpreting multiple call sites and extracting their corresponding parameter and taint propagation semantics within a single inference pass. By delegating the unified call-site inference to the model, we avoid issuing separate model queries for each individual call relation, thereby eliminating the interaction overhead inherent to traditional per-call analysis workflows.

Consequently, aggregating multiple call-site analyses into the unified context $\mathcal{M}(f_p)$ not only eliminates redundant code traversals but also substantially reduces the number of model-analysis interactions required to process complex firmware functions. This integration of call-site merging with joint LLM-based inference enables more scalable cross-call optimizations, such as shared variable inference and consolidated taint propagation tracking across multiple invocation paths within the same parent function.

D. Liva Multi-Agent

To analyze taint data connectivity within hazardous function call chains and precisely identify potential security vulnerabilities, we propose Liva, a novel multi-agent system comprising two specialized components: a taint flow tracking agent and a vulnerability inference agent. The taint flow tracking agent employs sophisticated variable dependency modeling to systematically trace taint data propagation paths from sources to arbitrary program points. Through comprehensive flow analysis, it constructs detailed taint propagation graphs that capture the complete data flow topology within the target program. Building upon these graphs, the vulnerability inference agent conducts security risk assessments by leveraging a library of predefined vulnerability patterns to systematically evaluate critical sink points, including system command execution functions and memory manipulation interfaces. This integrated approach enables highly accurate and efficient vulnerability identification across complex software systems.

1) *Taint Tracking Agent*: The upper part of Fig. 3 illustrates the workflow of this agent, which employs a large language model fine-tuned with taint-specific examples as its core analysis engine. Under a predefined task schedule, the agent processes function fragments and records parameter-passing relationships in a relational database. These records are subsequently queried by analytical tools to identify and output connected taint data flows.

a) *Fine-Tuning of Large Language Models*: To adapt large language models for taint flow analysis tasks, we employ LLaMaFactory [36] as our fine-tuning framework. We construct training datasets where each sample consists of code fragments

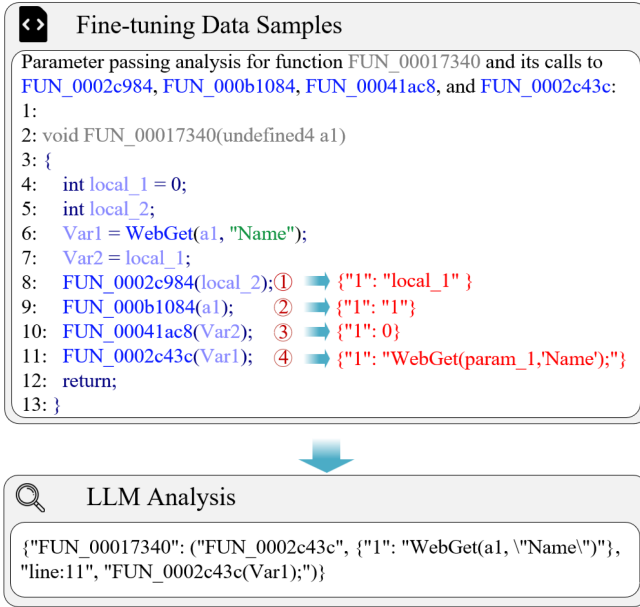


Fig. 4. Fine-tuning data with four parameter mapping relationships.

annotated with taint source information, paired with corresponding taint propagation patterns as target outputs.

The fine-tuning process leverages QLoRA implementation, which combines 4-bit quantization with Low-Rank Adaptation to achieve computational efficiency while maintaining model performance. This approach enables effective adaptation of pre-trained language models to understand taint propagation semantics through parameter-efficient training strategies.

b) Fine-Tuning data Samples: To enable the model to process code and interpret its analysis, the input and output formats were standardized. Each fine-tuning sample consists of a constructed input and its corresponding output, as illustrated in Fig. 4. The upper part of the figure depicts the input, which specifies the analysis objective and provides decompiled code annotated with line numbers, while the lower part presents the output formatted according to the parameter mapping structure described in Section IV-C2.

Through systematic analysis of real-world device firmware samples, we identified four distinct types of parameter mapping mechanisms: 1) local variable propagation, where parameters are transmitted through local variable assignments within function scopes; 2) formal parameter propagation, where parameters are passed through function argument lists following standard calling conventions; 3) constant value propagation, where parameters are transmitted as compile-time or runtime constants; and 4) return value propagation, where parameters are passed via function return values across call boundaries. As highlighted in red in Fig. 4, the model's mappings correspond directly to these four categories.

c) Storage and Querying of Relational Structures: We employ Neo4j [37] graph database to store parameter mapping relationships identified by our taint analysis intelligent agent. The graph schema consists of two node types: *Function Nodes* representing individual functions with their metadata,

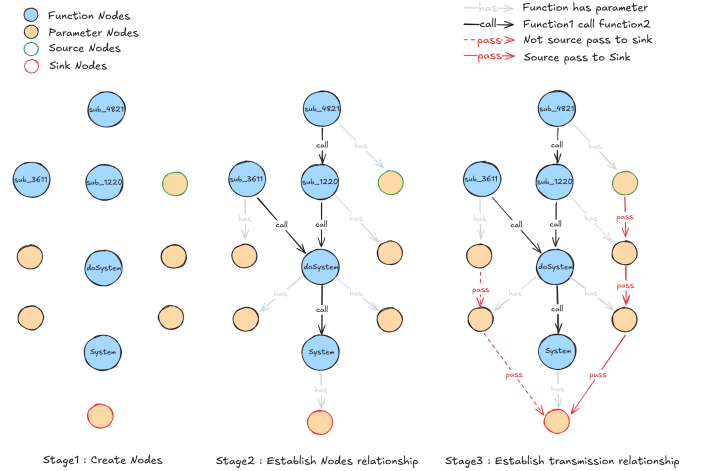


Fig. 5. Taint data storage and query. Storage includes function nodes, parameter nodes, and their relationships. Query focuses on connectivity analysis between sources and sinks.

and *Parameter Nodes* representing parameters and variables within function scopes. The relationships between nodes capture parameter mapping connections, where each edge represents data flow from caller to callee parameters.

Fig. 5 depicts the stored procedure, which is organized into three stages. In the first stage, function nodes and corresponding parameter nodes are constructed based on the Liva agent's output, and source and sink nodes are identified and labeled. The second stage establishes invocation relationships among function nodes as well as ownership associations between parameter nodes and their respective functions. In the third stage, transmission relationships between parameter nodes are added to represent the overall data flow path.

Connectivity of tainted data flows can then be queried using Neo4j query syntax. In Fig. 5, the solid red arrows in the third stage denote valid flows from source to sink, whereas the dashed red arrows represent invalid tainted flows where the starting point is not a source node.

2) Vulnerability Inference Agent: While taint flow tracking successfully identifies potential data propagation paths from sources to sinks, the mere existence of such paths does not guarantee the presence of exploitable vulnerabilities. Critical vulnerability assessment requires determining whether input data can exceed buffer boundaries, bypass validation mechanisms, or circumvent security controls. Many source-to-sink flows are protected by sanitization mechanisms, bounded by conditional guards, or rendered unexploitable through proper validation procedures. To address the distinction between theoretical data flows and actual security threats, we introduce a vulnerability inference agent using semantic analysis to filter false positives and confirm genuine vulnerabilities.

As illustrated in Fig. 6, the agent receives as input the connected taint flow chains identified by the tracking stage, along with the corresponding decompiled code fragments of all involved functions. Each candidate flow is represented as an ordered sequence $p = \langle f_s \rightarrow f_{i_1} \rightarrow f_{i_2} \rightarrow \dots \rightarrow f_t \rangle$, where

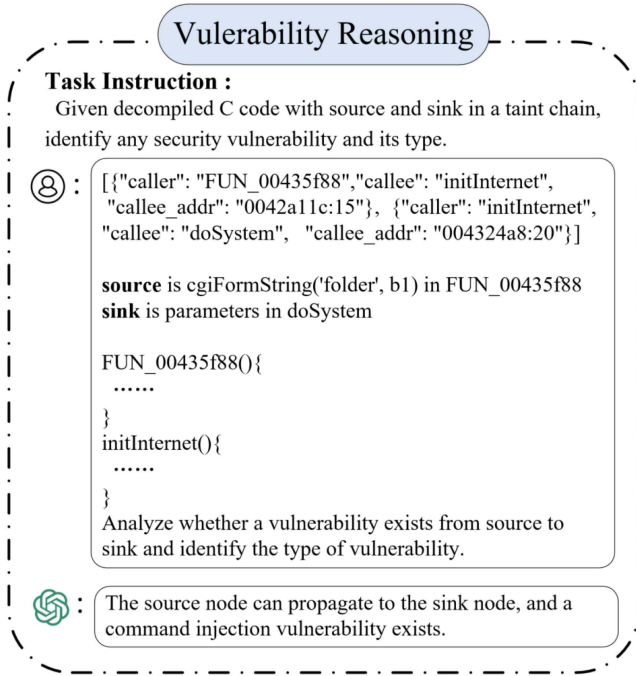


Fig. 6. Vulnerability inference prompt.

f_s denotes the taint source function, f_t represents the sink function, and $f_{i,j}$ are intermediate functions in the propagation path. The parameter mapping relations $\mathcal{M}_{i,j}$ captured during taint tracking provide precise variable correspondence across function boundaries, enabling comprehensive semantic analysis of the entire flow chain.

To systematically evaluate the exploitability of each candidate path, we employ a commercial LLM with carefully designed prompts that guide its inference. The framework directs the model to inspect each flow for vulnerability indicators, including unchecked input lengths, missing boundary validations, and dangerous function calls in critical paths.

The inference leverages the model's understanding of programming semantics and security principles to assess whether each candidate path $p \in \mathcal{P}_{\text{candidate}}$ is a genuine security threat. We define a binary judgment function $\mathcal{J} : \mathcal{P}_{\text{candidate}} \rightarrow 0, 1$ such that $\mathcal{J}(p) = 1$ indicates a confirmed vulnerability and $\mathcal{J}(p) = 0$ represents a benign flow.

This semantic inference approach directly addresses the core challenge of reducing false positives in static vulnerability detection. Traditional analysis tools not only flag dangerous sinks such as `strcpy()`, but also incorporate heuristic rules to detect whether buffer boundaries are checked. However, these approaches fail to capture full semantic context: boundary checks may exist but cannot guarantee safety under complex control flow, indirect dependencies, or weak validation. By contrast, our inference agent integrates the full call chain, variable mappings, and contextual guards into the evaluation. This allows it to distinguish genuinely exploitable flows—such as unchecked input reaching a call to `system()`—from those that are protected by effective boundary validations. Consequently,

the candidate set $\mathcal{P}_{\text{candidate}}$ is transformed into a refined alert set $\mathcal{P}_{\text{alert}} = p \in \mathcal{P}_{\text{candidate}} \mid \mathcal{J}(p) = 1$, significantly enhancing detection precision.

V. IMPLEMENTATION

The Liva prototype system was implemented with approximately 7000 lines of Python code and 2000 lines of Java code. It integrates IDA Pro 9.0 and Ghidra 10.4 to perform decompilation and function analysis on firmware binaries, and currently supports multiple architectures, including x86, ARM, MIPS, and PowerPC. For source identification, Liva leverages string indexing and disassembly features to infer functions related to HTTP user input and mark them as sources. For sink identification, the system preprocesses binaries to extract their dependent shared library functions, applies rule-based matching to identify potentially dangerous functions, and further refines their classification through intelligent-agent analysis.

At its core, Liva incorporates a LLM-based multi-agent system to support inter-procedural data-flow analysis and vulnerability inference. The taint analysis agent is built on a fine-tuned Qwen3-32B model, trained with 3,000 function samples extracted from real-world firmware to capture taint propagation patterns; its results are organized in a graph database for automated modeling of cross-function data-flow paths. The vulnerability inference agent employs a commercial LLM as its inference engine, leveraging taint propagation results, call graphs, and decompiled code to automatically infer potential vulnerabilities and classify their types (e.g., buffer overflow, command injection). By combining static analysis with LLM-driven multi-agent inference, Liva achieves efficient and automated vulnerability discovery across heterogeneous firmware.

VI. EVALUATE

In this section, we conducted our evaluation of Liva to answer the following research questions:

RQ1: How effectively does Liva identify sources and sinks within target code?

RQ2: How effective are Liva's top-down function flow analysis algorithm, predicated on source clustering, and its call chain merging algorithm?

RQ3: How are open-source large language models selected and fine-tuned in Liva? How do they compare with commercial models in taint analysis?

RQ4: How does Liva's performance compare to that of existing static taint analysis tools?

RQ5: How effective is Liva in discovering 0-day vulnerabilities in real-world IoT firmware?

A. Experiment Setup

Configuration: Liva was deployed on Ubuntu 22.04 LTS with an Intel Core i9-12900KF 3.20 GHz CPU (16 cores, 24 threads) and 16 GB DDR4 RAM. For large-scale model fine-tuning tasks, we utilized two NVIDIA RTX PRO 6000 Blackwell Server Edition GPUs, each equipped with 96 GB of ultra-fast GDDR7 memory, providing a total of 192 GB

TABLE I
PERFORMANCE OF LLMs ON SOURCE/SINK IDENTIFICATION (COST UNIT: USD
PER 1 M TOKENS)

Model	Cost	Source				Sink			
		TP	FP	Prec	Rec	TP	FP	Prec	Rec
DeepSeek-v3	\$0.14	10	0	100%	100%	10	1	91%	100%
GPT-4o	\$2.50	10	0	100%	100%	10	1	91%	100%
Qwen-Max	\$0.33	10	0	100%	100%	10	5	67%	100%
Claude-Max	\$3	10	0	100%	100%	10	7	59%	100%

Each task uses 10 positive and 10 negative samples.

GPU memory to handle the computational demands. The development environment consisted of Python 3.8, PyTorch 1.12.0 with distributed training support, and CUDA 12.8.

Firmware Dataset: To evaluate Liva, we constructed a multi-source firmware dataset. The samples were primarily collected from publicly available research datasets, including SATC, KARONTE, and FIRMAE, as well as from official vendor releases. As summarized in Table VII, the dataset comprises 64 firmware samples from 11 vendors, covering a wide range of device categories such as routers, access points, switches, VPN gateways, signal amplifiers, and network-attached storage systems. All samples embed web services and exhibit diversity in system architectures, implementation mechanisms, and routing strategies, thereby reflecting the heterogeneity of firmware design and security practices across different vendors.

Fine-tuning Dataset: To effectively fine-tune our model for vulnerability-oriented firmware analysis, we systematically constructed a dedicated dataset derived from real-world firmware samples containing known N-day vulnerabilities. Specifically, we first identified and extracted vulnerable call chains from the firmware collection and collected their inter-function parameter passing relationships as positive samples. To ensure balance during the training process, we further selected non-vulnerable functions from the same firmware codebase and extracted their parameter passing relationships as negative samples. Through this procedure, we constructed a total of 3,000 fine-tuning samples, covering diverse function types and execution contexts, thereby enhancing the model's adaptability to heterogeneous scenarios.

In addition, based on the four parameter mapping strategies described in Section IV-D1b, we constructed 400 test samples to systematically evaluate the model's generalization and robustness across different parameter transformation scenarios.

Baselines: We selected SATC [10], and Karonte [38] for comparison as our baselines. These techniques represent the state-of-the-art in firmware vulnerability analysis. SATC utilizes sources extracted from the frontend with predefined sink functions for taint analysis. Karonte focuses on identifying vulnerabilities caused by interactions between multiple binaries in firmware images. We did not include other static analysis tools due to their reliance on manual rule specification or limited applicability to firmware analysis scenarios.

Metrics: To evaluate the effectiveness of Liva in firmware vulnerability detection, we adopt five common classification metrics: *Alerts*, *True Positives (TP)*, *False Negatives (FN)*, *Precision*, and *Recall*. *Alerts* denotes the total number of vulnerability

warnings generated by the tool for a given firmware sample; *TP* represents the number of real vulnerabilities correctly identified by the tool; *FN* refers to the number of real vulnerabilities that the tool failed to detect. Based on these values, we further compute Precision and Recall, which are defined as $Precision = \frac{TP}{Alerts}$ and $Recall = \frac{TP}{TP+FN}$.

Precision measures the proportion of true vulnerabilities among all reported alerts, reflecting the accuracy of the detection results. Recall measures the proportion of real vulnerabilities successfully detected, reflecting the completeness of detection. In addition, we also record the running time (*Time*) of each tool on different firmware samples, in order to evaluate the efficiency of vulnerability testing.

B. RQ1: Source and Sink Identification

1) **LLM Selection for Identification:** To select an appropriate LLM for our source and sink identification pipeline, we first conducted a small comparative study using four representative models: DeepSeek-V3, GPT-4o, Qwen-Max, and Claude-Max. We collected function snippets from Tenda, D-Link, and Linksys firmware and prepared balanced datasets for both tasks (10 positive and 10 negative samples each). All models were queried with the same prompt template, which is included in the appendix A and B, available online.

The results are shown in Table I. While all four models perform equally well on source identification, their behavior differs noticeably on the sink task. DeepSeek-V3 and GPT-4o achieve the highest precision, each producing only a single false positive, whereas Qwen-Max and Claude-Max produce substantially more. Given that DeepSeek-V3 delivers accuracy comparable to GPT-4o but at a much lower inference cost, it offers the most practical balance for our setting. For this reason, we use DeepSeek-V3 as the LLM for source and sink identification in the remainder of the paper.

2) **Source Identification:** Table II presents a comparison of the number of sources and sinks identified by Liva and SaTC across 15 IoT firmware samples.

Liva identified a total of 6,723 sources, representing an 8.1% increase over the 6,217 detected by SaTC. In most cases, Liva outperformed SaTC, with the largest improvement observed in the DrayTek Vigor3900 (207.9%). However, for a few firmware samples such as Linksys WRT54 G, Tenda W15E, and Trendnet TEW755AP, Liva reported fewer sources. Manual inspection revealed that these decreases were due to numerous false positives in SaTC, which inflated its results. This indicates that Liva achieves higher precision by reducing spurious detections, leading to more reliable source identification overall.

3) **Sink Identification:** For sink identification, Liva detected 142 functions compared to 115 by SaTC, an increase of 23.5%. The most notable improvement occurred in the Trendnet TEW824DRU firmware, with a 75.0% gain. These improvements stem from Liva's ability to handle wrapper functions in deep call chains, where SaTC often failed to propagate data. In some firmware (e.g., TP-Link WPA8630, WavLink WN579A3, TOTOLink A3002R, Tenda W15E, and Trendnet TEW755AP), both tools produced identical results, suggesting simpler call

TABLE II
SOURCE AND SINK IDENTIFICATION COMPARISON BETWEEN LIVA AND SATC ON REPRESENTATIVE IOT DEVICES

Vendor	Device	Program	Type	Arch	Source			Sink		
					SaTC	Liva	Increase %	SaTC	Liva	Increase %
Cisco	RV110W	httpd	VPN	MIPS	225	422	87.6	7	9	28.6
D-Link	DI8300	jhttpd	Router	MIPS	371	526	41.8	8	11	37.5
D-Link	DNS320L	account_mgr.cgi	NAS	ARM	52	90	73.1	7	10	42.9
NetGear	R8500	httpd	Router	ARM	1263	1307	3.5	9	11	22.2
NetGear	R7900	htpd	Router	ARM	1135	1250	10.1	8	10	25.0
TP-Link	WPA8630	httpd	Powerline Kit	MIPS	142	146	2.8	8	8	-
WavLink	WN579A3	login.cgi	WiFi Extender	MIPS	14	13	-7.1	7	7	-
Belkin	N600	webs	WiFi Extender	MIPS	682	750	10.0	7	10	42.9
TOTOLink	A3002R	boa	WiFi Extender	MIPS	657	785	19.5	7	7	-
Linksys	WRT54G	httpd	Router	MIPS	331	175	-47.1	7	8	14.3
Trendnet	TEW824DRU	ssi	AP	MIPS	130	171	31.5	8	14	75.0
Trendnet	TEW755AP	ssi	AP	MIPS	58	38	-34.5	7	7	-
Tenda	AX1806	tdhttpd	Router	ARM	244	286	17.2	8	11	37.5
Tenda	W15E	httpd	Router	ARM	824	490	-40.5	7	7	-
DrayTek	Vigor3900	mainfunction.cgi	VPN	ARM	89	274	207.9	10	12	20.0
Total	-	-	-	-	6217	6723	8.1	115	142	23.5

structures. Overall, Liva provides broader and more accurate sink coverage, enhancing the completeness of vulnerability analysis.

4) *FP and FN Analysis*: For *source* identification, the LLM consistently and accurately locates functions related to user input, with no significant false negatives or false positives observed in the experiments. This is primarily due to the distinctive characteristics of such functions: on one hand, the function names often directly reflect the intent of capturing external inputs (e.g., `websGetVar`, `get_cgi`, `cgiGetValue`); on the other hand, the patterns for extracting user input from HTTP protocol structures are generally fixed, making it easier for the model to capture these patterns. However, false positives are more prominent for *sinks* encapsulated within shared libraries. These functions typically involve multiple layers of function calls, where parameters may be formatted or converted into non-string types during transmission. For example, in the `send_msg_to_netctrl` function of Tenda devices, the parameter `a1` is passed to the final `system` function using the `"%d"` format, which prevents the retention of the original string semantics, making it difficult for the LLM to determine whether it constitutes a real command injection point, leading to false positives.

C. RQ2: Function Flow Analysis Algorithm Evaluation

To mitigate redundant computation in taint analysis, a function aggregation strategy was employed. The evaluation was carried out along two dimensions: source aggregation and sensitive-path aggregation, with the detailed results summarized in Table III.

1) *Source Aggregation*: In the 15 router firmware samples, a total of 6,723 source points were identified. After applying aggregation, this number was reduced to 1,281, corresponding to an overall reduction of 80.95%. Most devices achieved an aggregation ratio between 70% and 90%. For instance, NetGear R8500 and TOTOLink A3002R reached 90.21% and 91.34%, respectively, while even a device with fewer sources, WavLink

TABLE III
SOURCE POINT AGGREGATION AND SENSITIVE FUNCTION MERGING RESULT

Vendor Device	Source Point	Merged	Analysis Count %	Sensitive Function	Merged	Analysis Count %	FN Count
Cisco RV110W	422	104	75.36	468	90	80.77	0
D-Link DI8300	526	84	84.03	426	293	31.22	1
D-Link DNS320L	90	32	64.44	213	85	60.09	0
NetGear R8500	1307	128	90.21	720	272	62.22	0
NetGear R7900	1250	127	89.84	753	287	61.89	0
TP-Link WPA8630	146	44	69.86	321	132	58.88	1
WavLink WN579A3	13	2	84.62	13	6	53.85	0
Belkin N600	750	88	88.27	706	123	82.58	0
TOTOLink A3002R	785	68	91.34	415	111	73.25	0
Linksys WRT54G	175	29	83.43	68	19	72.06	0
Trendnet TEW824DRU	171	55	67.84	104	63	39.42	0
Trendnet TEW755AP	38	22	42.11	74	45	39.19	0
Tenda AX1806	286	50	82.52	320	151	52.81	1
Tenda W15E	490	140	71.43	719	310	56.88	0
DrayTek Vigor3900	274	108	60.58	288	155	46.18	0
Total	6,723	1,281	80.95	5,534	2,132	61.48	

WN579A3, achieved 84.62%. These results demonstrate that the proposed method effectively eliminates redundant sources and significantly reduces the scale of analysis.

2) *Sensitive Path Aggregation*: For sensitive function paths, 5,534 candidate paths were initially detected. After aggregation, the number decreased to 2,132, yielding an overall reduction of 61.48%. The reduction ratios varied across devices. Belkin N600 exhibited a high reduction of 82.58%, whereas D-Link DI8300 achieved only 31.22%. This indicates that some firmware samples contain a large number of overlapping paths that can be merged, while others have more dispersed paths with lower aggregation potential. Overall, the method substantially reduces redundant path tracing and enhances the efficiency of taint analysis.

Sensitive Path Aggregation is conducted during the taint analysis phase to merge multiple sensitive paths within the same calling function for joint analysis. Compared with individual path analysis, the number of analysis invocations is greatly reduced after aggregation. However, a certain degree of false negatives is observed on several devices, including D-Link, TP-Link, and Tenda. After further inspection, we find that, as shown in Fig. 12 (Appendix D, available online), the false negatives sample contains two highly similar call sites within

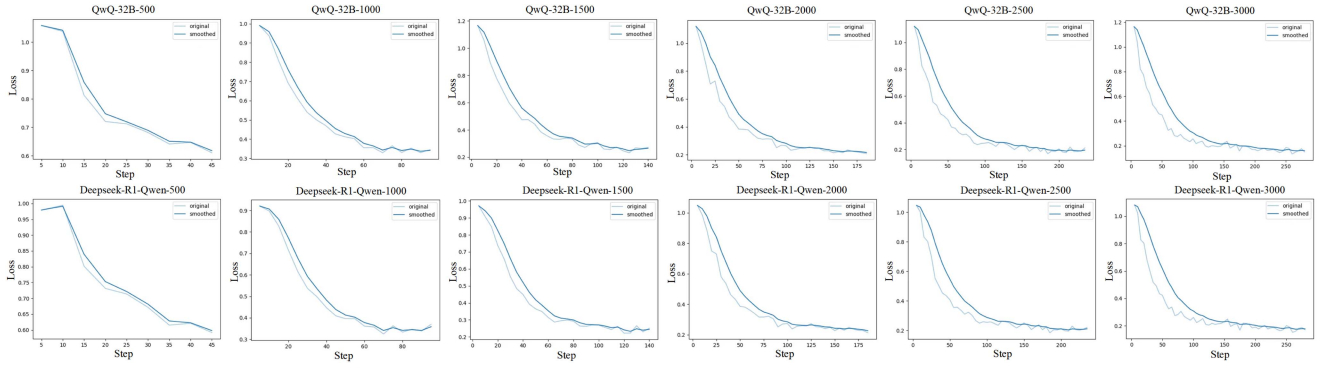


Fig. 7. Fine-tuning loss curves of QwQ-32B and DeepSeek-R1-Qwen under different data scales.

the same function. After aggregation, these two call sites are merged into a single analysis instance, which may affect the precision of the reported warnings.

D. RQ3: LLM Fine-Tuning Experiments

We selected four open-source large models with strong code analysis ability as the base models for fine-tuning: DeepSeek-R1-Distill-Qwen-32B [38], DeepSeek-Coder-33B-Instruct [39], [40], QwQ-32B [41], and Qwen3-32B [42]. In the following experiments, we will evaluate them to identify the most suitable model as the core of the Liva agent.

1) *Fine-Tuning Data Scale Determination*: To determine the proper data scale for fine-tuning, we carried out experiments on QwQ-32B and DeepSeek-R1-Qwen with training data ranging from 500 to 3000 samples, increasing by 500 each step. The results in Fig. 7 show that data scale has a direct impact on model convergence. With small data (500-1000 samples), the loss is high and the training curve is unstable. As the data size increases, convergence becomes faster, training curves become smoother, and the final loss decreases.

For QwQ-32B, Table IX (Appendix C, available online) shows that the loss is 1.1635 at step 8, drops quickly to 0.4503 at step 52, and reaches the lowest value of 0.1330 at step 262. When the data scale exceeds 2000, the loss continues to decrease but with reduced improvement. At 3000 samples, the loss is the lowest and the curve is the most stable, showing the best convergence. DeepSeek-R1-Qwen shows a similar pattern, where larger data improves both stability and convergence.

Therefore, considering convergence speed, final loss, and training stability, we selected 3000 samples as the default fine-tuning scale for the following experiments.

2) Open-Source Model Selection and Parameter Settings:

a) *Impact of Temperature on Accuracy*: Temperature parameters play a crucial role in regulating the creativity and diversity of content generated by large language models. To systematically evaluate the impact of temperature on Liva's analysis accuracy, we constructed a dataset containing 400 test samples covering the four typical parameter passing scenarios described in Section IV-D. Experiments were conducted on four fine-tuned open-source models, with temperature values set from 0.0 to 1.0 at 0.1 increments across 11 gradients. Each

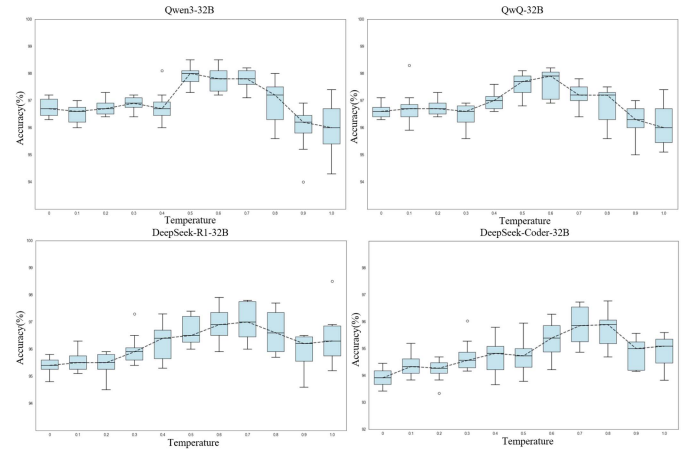


Fig. 8. Impact of temperature on analysis accuracy of fine-tuned open-source models.

experimental configuration was executed for 10 rounds to ensure statistical reliability of the results.

Fig. 8 illustrates the relationship between temperature parameters and model analysis accuracy. In the low-temperature range (0.0-0.4), model accuracy remained relatively low with limited variation, likely due to overly deterministic output patterns that constrained the parsing capability for complex parameter passing paths. As the temperature increased to the moderate range (0.5-0.7), accuracy significantly improved and reached respective peak values, indicating that models achieved enhanced exploratory capabilities while maintaining inference stability. As the temperature further entered the high-temperature range (0.8-1.0), accuracy began to exhibit notable fluctuations, suggesting that excessive randomness compromised the stability of code analysis.

Overall, a temperature of 0.5-0.7 provides the optimal balance between stability and detection for Liva.

b) *Impact of Models on Analysis Efficiency*: Fig. 8 reports the accuracy of the four open-source models, where Qwen3-32B and QwQ-32B achieved the highest average accuracies of 97.79% and 97.91%, respectively. Table V (upper panel) further compares their analysis time across the four categories of test data. The results show that Qwen3-32B consistently required

TABLE IV
TOKEN CONSUMPTION AND COST FOR DLINK DIR816 ANALYSIS

Model	Token Price (USD/1M)	Tokens Used	Cost (USD)
GPT-4o	\$2.50/\$10.00	632,880/2,782	\$1.61
Gemini-2.5-Pro	\$2.50/\$15.00	569,592/3,108	\$1.47
Qwen-Max	\$0.33/\$6.20	537,948/3,218	\$0.20
Claude-3.7-Sonnet	\$3.00/\$15.00	696,168/2,983	\$2.13
Liva	-/-	137,326/2,636	-

TABLE V
PERFORMANCE COMPARISON OF DIFFERENT DETECTION METHODS

Model	Type I Time	Type II Time	Type III Time	Type IV Time
Qwen3-32B	13:20	12:45	14:27	18:16
QwQ-32B	20:35 (↑7:15)	18:17 (↑5:32)	19:21 (↑4:54)	25:29 (↑7:13)
DeepSeek-R1-32B	18:41 (↑5:21)	17:51 (↑5:06)	18:25 (↑3:58)	24:48 (↑6:32)
DeepSeekCoder-33B	17:36 (↑4:16)	17:12 (↑4:27)	18:43 (↑4:16)	22:39 (↑4:23)
GPT-4o	14:56 (↑1:36)	15:04 (↑2:19)	16:48 (↑2:21)	17:41 (↓0:35)
Gemini-2.5-pro	16:28 (↑3:08)	15:11 (↑2:26)	17:26 (↑2:59)	21:53 (↑3:37)
Qwen-Max	15:43 (↑2:23)	13:37 (↑0:52)	12:38 (↓1:49)	19:34 (↑1:18)
Claude-3.7-Sonnet	14:17 (↑0:57)	12:39 (↓0:06)	16:12 (↑1:45)	17:36 (↓0:40)

the shortest processing time. In contrast, although QwQ-32B reached a comparable level of accuracy, its analysis time was substantially longer, with a maximum increase of approximately seven minutes relative to Qwen3-32B. The two DeepSeek models, DeepSeek-R1-32B and DeepSeek-Coder-33B, exhibited intermediate performance, but their processing times were generally 4-6 minutes longer than those of Qwen3-32B across all tasks.

Overall, Qwen3-32B provided the most favorable balance between accuracy and efficiency. QwQ-32B achieved high accuracy but with higher computational cost, while the two DeepSeek models yielded moderate results and did not surpass Qwen3-32B in either aspect. Based on these findings, Qwen3-32B was selected as the base model for fine-tuning.

3) *Commercial Model Comparison*: To comprehensively evaluate the performance of the proposed Liva model (fine-tuned from Qwen3-32B), we selected four mainstream commercial models as baselines, namely GPT-4o [43], Gemini-2.5-pro [44], Qwen-Max [45], and Claude-3.7-Sonnet [46]. The comparative experiments were conducted from two perspectives: accuracy and analysis time.

In terms of accuracy, Liva achieved leading performance on the Type-2, Type-3, and Type-4 datasets, with accuracies of 98.21%, 97.33%, and 92.58%, respectively (Fig. 9). It was only one percentage point lower than GPT-4o on Type-1, while matching or surpassing Claude-3.7-Sonnet in other cases, thus demonstrating consistently high accuracy overall. In terms of analysis efficiency, certain commercial models demonstrated faster processing speed in specific tasks, but this often came at the expense of accuracy. For example, on the Type-3 dataset, Qwen-Max reduced runtime by 1 minute 49 seconds compared to Liva, but its accuracy was only 93%, which is 4 percentage points lower than Liva. Similarly, GPT-4o was 35 seconds faster than Liva on the Type-4 dataset, yet its accuracy dropped to 86%, which is 6 percentage points lower.

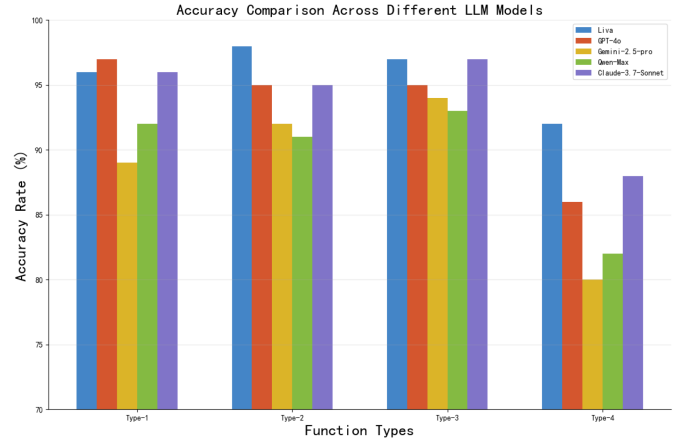


Fig. 9. Analysis accuracy comparison between Liva and commercial large models across four parameter-mapping datasets.

Beyond accuracy and runtime, we further examined the resource consumption associated with conducting taint analysis using different models. Using the D-Link DIR816 firmware as an example, the commercial LLMs incurred token usage in the range of 500k-600k tokens per binary. In contrast, Liva exhibited substantially reduced token consumption. This reduction stems from the fact that commercial LLMs typically require long prompts to explicitly specify task requirements during API-based interactions, whereas a fine-tuned model like Liva can directly interpret structured analysis inputs with minimal contextual guidance. Although the per-program cost of commercial models remains relatively low, and Liva's token price cannot be fully quantified due to its local deployment, a single firmware often contains multiple binaries requiring analysis. Consequently, when scaling to large firmware datasets, Liva offers a significant advantage in terms of reducing token usage and overall operational cost.

Overall, although Liva required slightly longer analysis time than some commercial models for certain tasks, it consistently maintained higher accuracy, thereby demonstrating superior practical effectiveness in supporting taint analysis.

4) *Impact of Decompilation Quality on Analysis Accuracy*: Since the analysis heavily relies on the accuracy of decompiled code, decompilation errors may potentially affect the precision of taint analysis results. To evaluate the robustness under such errors, the impact introduced by different decompilation tools is further investigated. Compared with IDA, Ghidra is more prone to decompilation errors in control-flow recovery, type and pointer inference, and variable reconstruction, which may significantly distort both intra- and inter-procedural taint propagation paths. For each of these three categories, 10 representative samples are selected from the decompiled code generated by Ghidra, and analyzed together with the corresponding decompilation results produced by IDA using the Liva taint analysis framework. The comparative results are summarized in Table VI.

The results indicate that the lower quality of Ghidra's decompiled code, compared with IDA, leads to a significant reduction in taint analysis accuracy across the three error categories.

TABLE VI
ACCURACY COMPARISON OF DIFFERENT DECOMPILERS UNDER THREE
TYPICAL ERROR CATEGORIES

Decompiler	Control-Flow Recovery	Type & Pointer Inference	Variable Recovery
IDA	10	10	10
Ghidra	9	6	5

Control-flow recovery errors have a limited impact, whereas type inference and variable reconstruction errors have the most pronounced effect, reducing the accuracy to only 50% -60%. This suggests that deficiencies in type and variable recovery in Ghidra are the primary factors affecting the analysis precision.

E. RQ4: Comparison With Baseline

Table VII summarizes the comprehensive comparison between Liva and the baseline systems SaTC and Karonte on 65 firmware samples collected from 11 major IoT vendors, covering 370 confirmed vulnerabilities, including 64 zero-day vulnerabilities. The results show that Liva outperforms the baselines across multiple metrics.

1) *Detection Effectiveness*: On the evaluation dataset, Liva achieved an average recall of 98.1%, successfully identifying 363 out of the 370 confirmed vulnerabilities with only 6 misses. In contrast, SaTC reached a recall of only 14.6% (316 misses), while Karonte achieved a recall of merely 3.8% (356 misses), both suffering from severe under-coverage. Moreover, Liva maintained stable performance across different vendors, indicating its robustness against variations in firmware implementations and architectures.

2) *Precision Analysis*: While maintaining a high recall, Liva also achieved a precision of 74.6%, with 487 total alerts, significantly outperforming SaTC (40.3%, 134 alerts) and Karonte (21.5%, 65 alerts). This shows that Liva's multi-agent architecture is effective not only in achieving comprehensive vulnerability coverage—including a substantial number of zero-day vulnerabilities—but also in reducing false positives.

3) *Computational Efficiency*: Liva completed the entire analysis in 4,139 minutes (68.9 hours), whereas SaTC and Karonte required 27,161 minutes (452.7 hours) and 22,632 minutes (377.2 hours), respectively. Thus, Liva achieved approximately $6.7 \times$ and $5.6 \times$ speedups over the two baselines. These results highlight that Liva is capable of significantly reducing analysis cost while preserving high detection accuracy.

4) *FP and FN Analysis for Liva*: In the analysis of false positives, we found that Cisco (62.5%), Linksys (64.7%), and D-Link (69.4%) exhibited relatively low precision, mainly due to three reasons. First, some D-Link firmware includes the command injection filtering function SpecSymbol2BackSlash, which escapes special characters. Although the filter can theoretically be bypassed using %0a, this character cannot be introduced when the source function is cgiFormStringNoNewlines, leading to cases being incorrectly flagged as vulnerable during analysis. Second, in Cisco and Linksys devices, certain source functions lack valid routing paths, causing taint analysis to

fail to propagate, yet these functions were still misidentified as vulnerable. Third, when handling HTTP parameters, Liva may trigger secondary routing. Since alarms are raised at the source level, the same source point can be reused across different routing paths, resulting in duplicate alerts. These factors directly cause the high FP rates in these devices.

In analyzing false negatives, we reviewed six cases from Netgear, TP-Link, Linksys, and Tenda. The primary issue lies in Liva's inability to recognize cross-function data storage and retrieval, which prevents the correct modeling of taint propagation and leads to missed detections. In addition, in Belkin's CVE-2025-7094, the formBSSetSitesurvey function produced an excessively long decompiled body. During code reduction, key data propagation statements were omitted, causing taint analysis to fail.

5) *FP and FN Analysis for SaTC and Karonte*: The false positives of SaTC mainly originate from inaccurate source identification within backend binaries. A considerable number of the identified sources do not correspond to actual routing paths, thereby generating spurious alerts.

In contrast, the false negatives of Karonte are primarily attributed to its emphasis on inter-binary data propagation, while neglecting user inputs derived from HTTP requests. Consequently, some critical input sources remain undetected.

Moreover, both tools share common causes of false negatives. First, their source identification is incomplete, making it difficult to capture sources that lack corresponding frontend code. Second, both tools encounter difficulties in effectively recognizing wrapper functions of dangerous calls within dynamically linked libraries, leading to missed detections.

F. RQ5: Discovery of 0-Day Vulnerabilities

As shown in Table VIII, Liva successfully discovered 64 previously unknown (0-day) vulnerabilities across 15 different IoT device models, most of which involve command injection (CI) and memory corruption. These vulnerabilities were reported to the corresponding vendors and confirmed, with 39 already assigned official CVE or CNVD identifiers. The results demonstrate that Liva can effectively detect vulnerabilities in real-world IoT firmware and uncover security issues that baseline methods fail to identify, highlighting strong generalization capability in practical scenarios. Further analysis indicates that the primary reasons for the baselines' omissions can be summarized as follows: 1) incomplete identification of sources or sinks, and 2) inability to capture inter-variable relationships during taint analysis.

VII. RELATED WORK

A. Taint Analysis

Taint analysis is a program security analysis technique that tracks untrusted data flow from source points to sensitive operation points (sinks) to detect potential security vulnerabilities [5]. In source code analysis, FlowDroid [47] and TAJIS [48] achieve precise taint tracking by building control flow graphs and data flow graphs. For web applications, RIPS [49] and

TABLE VII
VULNERABILITY DETECTION PERFORMANCE OF LIVA, SATC AND KARONTE ON IOT FIRMWARE(TIME UNIT: MINUTES)

Vendor	Device	Samples	Vuln	Liva						SaTC						Karonte					
				Alerts	TP	FN	Prec(%)	Recal(%)	Time	Alerts	TP	FN	Prec(%)	Recal(%)	Time	Alerts	TP	FN	Prec(%)	Recal(%)	Time
Cisco	RV	1	5(4)	8	5	0	62.5%	100.0%	35	7	1	4	14.3%	20.0%	197	4	1	4	25.0%	20.0%	133
D-Link	DIR/DNS/DCS/DI	10	59(33)	85	59	0	69.4%	100.0%	633	21	6	53	28.6%	10.7%	4,135	17	3	56	17.7%	5.1%	3,373
NetGear	WNR/RAC	14	12(2)	13	11	1	84.6%	91.7%	789	9	2	10	22.2%	16.7%	8,193	11	3	9	27.3%	25.0%	6,211
TP-Link	WR/WPA	5	10(2)	12	9	1	75.0%	90.0%	366	3	2	8	66.7%	20.0%	1,126	1	1	9	100.0%	10.0%	928
WavLink	WN	2	23(4)	27	23	0	85.2%	100.0%	46	8	1	22	25.0%	4.3%	566	4	0	23	0.0%	0.0%	781
Belkin	N	1	15(1)	16	14	1	87.5%	93.3%	68	7	3	12	42.9%	20.0%	256	7	2	13	28.6%	13.3%	389
Totolink	X/A/N/C/E/CS	10	82(2)	113	82	0	72.6%	100.0%	625	18	5	77	27.8%	6.1%	4,757	0	0	82	0.0%	0.0%	817
Linksys	WRT/RE/E	3	22(4)	34	22	0	64.7%	100.0%	221	0	0	22	0.0%	0.0%	173	3	0	22	0.0%	0.0%	693
Trendnet	TEW	4	27(4)	32	26	1	81.3%	96.3%	318	9	3	24	33.3%	11.1%	1,847	5	1	26	20.0%	3.7%	1,819
Tenda	AC/AX/FH/W	11	96(7)	126	93	3	73.8%	96.9%	737	49	31	65	46.2%	63.3%	4,946	8	3	93	37.5%	3.1%	5,760
DrayTek	V	3	19(1)	21	19	0	90.5%	100.0%	301	13	0	19	0.0%	0.0%	965	5	0	19	0.0%	0.0%	1,728
Total	-	64	370(64)	487	363	7	74.6%	98.1%	4,139	144	54	316	37.5%	14.6%	27,161	65	14	356	21.5%	3.8%	22,632

TABLE VIII
ZERO-DAY VULNERABILITIES DISCOVERED BY LIVA IN FIRMWARE DATASET

Vendor & Model	Number	Unknown Vulnerabilities
Cisco RV110W	4	4 unassigned
D-Link DI8300	6	CNVD-2024-48723 to CNVD-2024-48725, CNVD-2024-49420 to CNVD-2024-49422
D-Link DNS320L	27	CVE-2024-7828 to CVE-2024-7832, CVE-2024-7715, CVE-2024-7849, CVE-2024-8127 to CVE-2024-8134, CVE-2024-8210 to CVE-2024-8214, CVE-2024-7922, CNVD-2024-47670 to CNVD-2024-47675
NetGear R8500	1	CNVD-2024-47646
NetGear R7900	1	1 unassigned
TP-Link WPA8630	2	2 unassigned
WavLink WN579A3	4	CNVD-2025-18059, CNVD-2025-18060, 2 unassigned
Belkin N600	1	1 unassigned
Totolink A3002R	2	2 unassigned
Linksys WRT54G	4	CVE-2024-41281, CVE-2024-8408, 2 unassigned
Trendnet TEW824DRU	1	1 unassigned
Trendnet TEW755AP	3	3 unassigned
Tenda AX1806	2	CNVD-2024-47661, 1 unassigned
Tenda W15E	5	5 unassigned
DrayTek Vigor3900	1	1 unassigned
Total	64	

Pixy [50] build taint propagation models to detect vulnerabilities like XSS and SQL injection. In mobile applications, TaintDroid [51] implements Android system-level dynamic taint tracking, while IccTA [52] focuses on taint propagation analysis for inter-component communication. However, taint analysis for IoT device binary programs faces greater challenges. The main problems include missing program structure information, complex indirect jump handling, and code optimization obfuscation [53]. These issues usually require symbolic execution techniques for resolution [54].

B. Static Analysis in Firmware

IoT firmware typically appears as closed-source binaries lacking source code and debug information. Traditional code auditing and dynamic testing methods become difficult to apply. In contrast, static binary analysis works without runtime environments, making it important for firmware security. DTaint [8] implements taint tracking through function-level data dependency graphs and uses bottom-up call graph analysis to reduce complexity. However, this method shows poor precision when handling complex control flows and function pointers. The heuristic-based source and sink identification provides limited coverage. SATC [13] identifies input sources using shared keywords and locates potential source points by analyzing keyword patterns in communication interfaces. However, this approach relies heavily on string matching and performs poorly in code obfuscation or dynamic interface scenarios. Karonte [12] focuses on multi-binary interactions and can model data flow and control flow to detect cross-component vulnerabilities. However, the method remains rough in fine-grained propagation analysis within single binaries.

HermeScan [55] reduces redundant computation through progressive propagation strategies and improves efficiency. However, the method relies on predefined rules and struggles with new code patterns and compiler optimizations.

C. LLM-Assisted Analysis

In recent years, researchers have actively explored combining the code parsing abilities of large language models with traditional program analysis techniques to enhance code auditing effectiveness. LLMDFA [56] proposes a data flow analysis framework supported by large language models, exploring application ideas in source code analysis. LATTE [26] proposes the first static binary taint analysis method driven by large language models without requiring manual customization of taint propagation rules. IRIS [57] proposes a neural-symbolic combination approach that systematically integrates large language models with static analysis techniques to detect complex security vulnerabilities requiring whole-repository analysis. MFT-Coder [58] enhances code large language model capabilities through multi-task fine-tuning frameworks, proving that joint training has significant advantages over independent fine-tuning. Vul-RAG [59] designs a knowledge-level retrieval-augmented generation (RAG) framework [60] that further improves vulnerability identification accuracy.

To address challenges in IoT firmware binary static taint analysis, we propose the Liva prototype. Liva uses large model multi-agent collaborative mechanisms to enhance understanding of firmware code semantics and functions. The system can automatically identify source and sink points and analyze parameter passing relationships between functions, assisting taint analysis and automated vulnerability determination.

VIII. CONCLUSION

This paper presents a LLM multi-agent solution, Liva, designed for taint analysis of IoT firmware binaries. The approach addresses three major limitations of traditional taint analyzers: (i) insufficient accuracy in identifying sources and sinks, often resulting in false positives and false negatives; (ii) excessive redundant computation caused by complex function call relationships, leading to low efficiency; and (iii) heavy reliance on expert knowledge to define taint propagation rules, resulting in incomplete coverage. Liva leverages large language models to assist in identifying sources and sinks, and employs fine-tuned models to analyze the propagation of taint data across functions within dangerous call chains at the semantic level. The

system further utilizes a graph database for efficient storage and querying, with the retrieved connected paths subsequently evaluated by a dedicated vulnerability inference agent. Experimental results demonstrate that Liva achieves notable advantages in detecting taint-related vulnerabilities in IoT firmware. It not only reduces time cost but also improves detection accuracy. On the benchmark dataset, Liva successfully discovered 64 previously unknown vulnerabilities, of which 39 have been officially assigned CVE/CNVD identifiers. Compared with baseline tools, Liva identified more vulnerabilities in less time while significantly reducing false positives. Moreover, the implementation of Liva highlights the feasibility and practical value of applying fine-tuned small-parameter large models to taint analysis tasks.

REFERENCES

- [1] L. S. Vailshery, "Internet of Things (IoT) and non-IoT active device connections worldwide from 2010 to 2025," 2025. [Online]. Available: <https://www.statista.com/statistics/1101442/iot-number-of-connected-devices-worldwide/>
- [2] C. P. S. T. Ltd., Check point software's 2025 security report finds alarming 44% increase in cyber-attacks, 2025. [Online]. Available: <https://www.checkpoint.com/press-releases/check-point-software-2025-security-report-finds-alarming-44-increase-in-cyber-attacks-amid-maturing-cyber-threat-ecosystem/>
- [3] C. P. Research, 6 cyber security challenges emerge from world economic forum: 2025 threats, 2025. [Online]. Available: <https://blog.checkpoint.com/security/6-cyber-security-challenges-emerge-from-world-economic-forum-check-point-research-on-2025-threats/>
- [4] X. T. Feng, X. G. Zhu, Q.-L. Han, W. Zhou, S. Wen, and Y. Xiang, "Detecting vulnerability on IoT device firmware: A survey," *IEEE/CAA J. Automatica Sinica*, vol. 10, no. 1, pp. 25–41, Jan. 2023.
- [5] T. Bakhshi, B. Ghita, and I. Kuzminykh, "A review of IoT firmware vulnerabilities and auditing techniques," *Sensors*, vol. 24, no. 2, Jan. 2024, Art. no. 708.
- [6] M. Kim, D. Kim, E. Kim, S. Kim, Y. Jang, and Y. Kim, "FirmAE: Towards large-scale emulation of IoT firmware for dynamic analysis," in *Proc. 36th Annu. Comput. Secur. Appl. Conf.*, 2020, pp. 733–745.
- [7] T. Yavuz and C. Brant, "Security analysis of IoT frameworks using static taint analysis," in *Proc. 12th ACM Conf. Data Appl. Secur. Privacy*, Baltimore MD USA, 2022, pp. 203–213.
- [8] K. Cheng et al., "DTaint: Detecting the taint-style vulnerability in embedded device firmware," in *Proc. 48th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, Luxembourg City, 2018, pp. 430–441.
- [9] X. Wang, J. Zhai, and H. Yang, "Detecting command injection attacks in web applications based on novel deep learning methods," *Sci. Rep.*, vol. 14, no. 1, Oct. 2024, Art. no. 25487.
- [10] S. Chen, Z. Lin, and Y. Zhang, "SelectiveTaint: Efficient data flow tracking with static binary rewriting," in *Proc. 30th USENIX Secur. Symp.*, USENIX Association, 2021, pp. 1665–1682.
- [11] P. Liu et al., "Fits: Inferring intermediate taint sources for effective vulnerability analysis of IoT device firmware," in *Proc. 28th ACM Int. Conf. Architectural Support Programm. Lang. Operating Syst.*, New York, NY, USA, 2024, pp. 138–152.
- [12] N. Redini et al., "Karonte: Detecting insecure multi-binary interactions in embedded firmware," in *Proc. IEEE Symp. Secur. Privacy*, San Francisco, CA, USA, 2020, pp. 1544–1561.
- [13] L. Chen et al., "Sharing more and checking less: Leveraging common input keywords to detect bugs in embedded systems," in *Proc. 30th USENIX Secur. Symp.*, USENIX Association, 2021, pp. 303–319.
- [14] K. Cheng et al., "Detecting vulnerabilities in linux-based embedded firmware with SSE-Based on-demand alias analysis," in *Proc. 32nd ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, Seattle WA USA, 2023, pp. 360–372.
- [15] A. Qasem, M. Debbabi, and A. Soeanu, "Octopustaint: Advanced data flow analysis for detecting taint-based vulnerabilities in IoT/IoT firmware," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, New York, NY, USA, 2024, pp. 2355–2369.
- [16] J. Ye, X. Fei, X. d. C. de Carnavalet, L. Zhao, L. Wu, and M. Zhang, "Detecting command injection vulnerabilities in linux-based embedded firmware with LLM-based taint analysis of library functions," *Comput. Secur.*, vol. 144, Sep. 2024, Art. no. 103971.
- [17] T. Bakhshi, B. Ghita, and I. Kuzminykh, "A review of IoT firmware vulnerabilities and auditing techniques," *Sensors*, vol. 24, no. 2, 2024, Art. no. 708.
- [18] Y. Gui, D. He, and J. Xue, "Merge-replay: Efficient IFDS-based taint analysis by consolidating equivalent value flows," in *Proc. 38th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2024, pp. 319–331.
- [19] K. Hough and J. Bell, "A practical approach for dynamic taint tracking with control-flow relationships," *ACM Trans. Softw. Eng. Methodol.*, vol. 31, no. 2, Dec. 2021, Art. no. 26.
- [20] X. Li, Q. Wei, Z. Wu, and W. Guo, "Finding taint-style vulnerabilities in lua application of IoT firmware with progressive static analysis," *Appl. Sci.*, vol. 13, no. 17, 2023, Art. no. 9710.
- [21] Hex-Rays SA, "IDA pro: Interactive disassembler and debugger," 2024. [Online]. Available: <https://hex-rays.com/ida-pro/>
- [22] NSA Directorate Research, "Ghidra: A software reverse engineering framework," *Open Source Softw.*, 2019. [Online]. Available: <https://www.nsa.gov/resources/everyone/ghidra/>
- [23] H. Xiao et al., "Accurate and efficient recurring vulnerability detection for IoT firmware," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, New York, NY, USA, 2024, pp. 3317–3331.
- [24] W. Gibbs et al., "Operation mango: Scalable discovery of taint-style vulnerabilities in binary firmware services," in *Proc. 33rd USENIX Secur. Symp.*, Philadelphia, PA, USENIX Association, 2024, pp. 7123–7139.
- [25] J. Zhao et al., "Leveraging semantic relations in code and data to enhance taint analysis of embedded systems," in *Proc. 33rd USENIX Secur. Symp.*, Philadelphia, PA, USENIX Association, Aug. 2024, pp. 7067–7084.
- [26] P. Liu et al., "LLM-powered static binary taint analysis," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 3, Feb. 2025, Art. no. 83.
- [27] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, Curran Associates, Inc., 2020, pp. 1877–1901.
- [28] A. Chowdhery et al., "PaLM: Scaling language modeling with pathways," *J. Mach. Learn. Res.*, vol. 24, no. 1, Jan. 2023, Art. no. 240.
- [29] X. Du et al., "Evaluating large language models in class-level code generation," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng.*, New York, NY, USA, 2024, pp. 982–994.
- [30] Q. Feng, R. Zhou, C. Xu, Y. Cheng, B. Testa, and H. Yin, "Scalable graph-based bug search for firmware images," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, New York, NY, USA, 2016, pp. 480–491.
- [31] P. Ferrara, A. K. Mandal, A. Cortesi, and F. Spoto, "Static analysis for discovering IoT vulnerabilities," *Int. J. Softw. Tools Technol. Trans.*, vol. 23, no. 1, pp. 71–88, Feb. 2021.
- [32] I. Gotovchits, R. Van Tonder, and D. Brumley, "Saluki: Finding taint-style vulnerabilities with static property checking," in *Proc. Workshop Binary Anal. Res.*, Internet Society, San Diego, CA, 2018, pp. 1–14.
- [33] G. D. Vito, F. Palomba, and F. Ferrucci, "The role of large language models in addressing IoT challenges: A systematic literature review," *Future Gener. Comput. Syst.*, vol. 171, 2025, Art. no. 107829.
- [34] X. Zhou, S. Cao, X. Sun, and D. Lo, "Large language model for vulnerability detection and repair: Literature review and the road ahead," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 5, pp. 1–31, May 2025.
- [35] A. Shestov, R. Levichev, R. Mussabayev, E. Maslov, A. Cheshkov, and P. Zadorozhny, "Finetuning large language models for vulnerability detection," 2024, *arXiv:2401.17010*. [Online]. Available: <https://arxiv.org/abs/2401.17010>
- [36] Y. Zheng et al., "LlamaFactory: Unified efficient fine-tuning of 100 language models," Jun. 2024, *arXiv:2403.13372*.
- [37] Neo4j Inc., "Neo4j graph database platform," *Graph Database Management System, Version 5.x*, 2024. [Online]. Available: <https://neo4j.com>
- [38] DeepSeek-AI, "Deepseek-R1-distill-qwen-32B," 2024, Accessed: Aug. 20, 2025. [Online]. Available: <https://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Qwen-32B>
- [39] DeepSeek-AI, "deepseek-coder-33b-instruct," 2024, Accessed: Aug. 20, 2025. [Online]. Available: <https://huggingface.co/deepseek-ai/deepseek-coder-33b-instruct>
- [40] D. Guo et al., "Deepseek-coder: When the large language model meets programming—the rise of code intelligence," 2024, *arXiv:2401.14196*. [Online]. Available: <https://arxiv.org/abs/2401.14196>
- [41] Qwen Team, "QwQ-32B," 2025, Accessed: Aug. 20, 2025. [Online]. Available: <https://huggingface.co/Qwen/QwQ-32B>
- [42] Q. Team, "Qwen3 technical report," 2025, *arXiv:2505.09388*. [Online]. Available: <https://arxiv.org/abs/2505.09388>

- [43] OpenAI, “GPT-4o: Omni-modal AI model,” 2024. [Online]. Available: <https://openai.com/index/hello-gpt-4o/>
- [44] Google DeepMind, “Gemini 2.5 pro: Our most intelligent AI model,” 2025, [Online]. Available: <https://blog.google/technology/google-deepmind/gemini-model-thinking-updates-march-2025/>
- [45] Qwen Team, “Qwen 2.5-max: Exploring the intelligence of large-scale moe,” 2025, Accessed: Aug. 20, 2025. [Online]. Available: <https://qwenlm.github.io/blog/qwen2.5-max/>
- [46] Anthropic, “Claude 3.7 Sonnet system card,” *System card (PDF) by Anthropic*, 2025. [Online]. <https://www.anthropic.com/claude-3-7-sonnet-system-card>
- [47] S. Arzt et al., “FlowDroid: Precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for android apps,” in *Proc. 35th ACM SIGPLAN Conf. Program. Lang. Des. Implementation*, Edinburgh U.K., 2014, pp. 259–269.
- [48] S. H. Jensen, A. Möller, and P. Thiemann, “Type analysis for javascript,” in *Proc. 16th Int. Symp. Static Anal.*, Berlin, Heidelberg, Springer-Verlag, 2009, pp. 238–255.
- [49] J. Dahse, “RIPS - A static source code analyser for vulnerabilities in PHP scripts,” in *Proc. Seminar Work (Seminer Çalışması)*, Horst Görtz Institute Ruhr-Univ. Bochum, Bochum, Germany, 2010.
- [50] N. Jovanovic, C. Kruegel, and E. Kirda, “Pixy: A static analysis tool for detecting web application vulnerabilities,” in *Proc. IEEE Symp. Secur. Privacy*, Berkeley/Oakland, CA, 2006, pp. 258–263.
- [51] W. Enck et al., “TaintDroid: An information-flow tracking system for realtime privacy monitoring on smartphones,” in *Proc. 9th USENIX Symp. Operating Syst. Des. Implementation*, Vancouver, BC, USENIX Association, 2010, pp. 393–407.
- [52] L. Li et al., “IccTA: Detecting inter-component privacy leaks in Android apps,” in *Proc. IEEE/ACM 37th IEEE Int. Conf. Softw. Eng.*, 2015, pp. 280–291.
- [53] Y. Shoshitaishvili et al., “SoL: (State of) the art of war: Offensive techniques in binary analysis,” in *Proc. IEEE Symp. Secur. Privacy*, 2016, pp. 138–157.
- [54] B. Yadegari, B. Johannesmeyer, B. Whitely, and S. Debray, “A generic approach to automatic deobfuscation of executable code,” in *Proc. IEEE Symp. Secur. Privacy*, 2015, pp. 674–691.
- [55] Z. Gao et al., “Faster and better: Detecting vulnerabilities in linux-based IoT firmware with optimized reaching definition analysis,” in *Proc. Netw. Distrib. Syst. Secur. Symp.*, vol. 26, 2024, pp. 1–16.
- [56] C. Wang, W. Zhang, Z. Su, X. Xu, X. Xie, and X. Zhang, “LLMDFA: Analyzing dataflow in code with large language models,” in *Proc. 38th Int. Conf. Neural Inf. Process. Syst.*, Red Hook, NY, USA, Curran Associates Inc., 2025.
- [57] Z. Li, S. Dutta, and M. Naik, “IRIS: LLM-assisted static analysis for detecting security vulnerabilities,” in *Proc. 13th Int. Conf. Learn. Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=9LdJDU7E91>
- [58] B. Liu et al., “MFTCoder: Boosting code LLMs with multitask fine-tuning,” in *Proc. 30th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, New York, NY, USA, 2024, pp. 5430–5441.
- [59] X. Du et al., “Vul-RAG: Enhancing LLM-based vulnerability detection via knowledge-level RAG,” 2025, *arXiv:2406.11147*. [Online]. Available: <https://arxiv.org/abs/2406.11147>
- [60] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. 34th Int. Conf. Neural Inf. Process. Syst.*, Red Hook, NY, USA, Curran Associates Inc., 2020, pp. 9459–9474.

Zhe Yang received the BS degree from Information Engineering University. He is currently working toward the MS degree with the School of Cyber Science and Technology, Beihang University. His research interests include IoT device security analysis and large language model system applications.

Hao Peng (Senior Member, IEEE) is currently a professor with the School of Cyber Science and Technology, Beihang University. His research interests include network content security and large language model system applications. He is the associate editor of *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, *Neural Networks*, and *International Journal of Machine Learning and Cybernetics*.

Yanling Jiang is currently a research professor with the School of Cyber Science and Technology, Beihang University. Her current research interests include cybersecurity and cyber Governance.

Jianwei Liu (Senior Member, IEEE) received the BS and MS degrees in electronic engineering from Shandong University, China, in 1985 and 1988, respectively, and the PhD degree in communication engineering from Xidian University, China, in 1998. He is currently a professor with the School of Cyber Science and Technology, Beihang University. His current research interests include cryptographic protocol design, security on wireless and mobile network, computer network security and Cryptography.

Hongbin Luo received the BS degree from Beihang University, in 1999 and the MS (with honors) and PhD degrees in communications and information science from the University of Electronic Science and Technology of China (UESTC), in June 2004 and March 2007, respectively. He is currently a professor and dean with the School of Cyber Science and Technology, Beihang University. His research interests are in the wide areas of network technologies including cybersecurity, routing and Internet architecture.

Mingsheng Tang (Member, IEEE) received the BS degree in computer science and technology from Xi'an Jiaotong University, in 2008, and the MS and PhD degrees in computer science and technology from the National University of Defense Technology, in 2010 and 2015, respectively. He currently is an associate professor with the School of Cyber Science and Technology, Beihang University. His current research interests include cybersecurity and multi-agents system.

Jiahe Li received the BS degree from Information Engineering University. He is currently working toward the MS degree with the School of Cyber Science and Technology, Beihang University. His research interests include cybersecurity and large language model system applications.

Kun Zhang received the BS degree from the National University of Defense Technology. He is currently working toward the MS degree with Beihang University. His research interests include applications related to multi-agent and large language model systems.